

Workerbee

[Overview](#)

[Motivation](#)

[Design](#)

[Workflow逻辑视图](#)

[Workflow节点](#)

[Workflow编辑](#)

[新建Workflow](#)

[编辑Workflow](#)

[删除Workflow](#)

[Workflow管理](#)

[可选客户端](#)

[不同harness接入](#)

[API配置](#)

[Base Assitant管理](#)

[额外工具管理](#)

[Concern](#)

Overview

Workerbee是一个集成多agent harness的跨平台高度客制化多agent协作workflow框架。软件旨在为用户提供一个基于本地任务场景定制的可编辑拓扑图结构的workflow，包括agent节点配置、客制化skills、不同harness集成、后台任务流监控、前端workflow拓扑图编辑、任务流水线历史与细粒度监控等功能。

Motivation

在日常学习工作中，我们会遇到这样一种需求：一个长程任务中，涉及项目观察、理解、设计、规划任务流程、具体任务节点实施、代码审计与测试、文档编写等一系列具有明确

逻辑上的先后依赖关系的流程。

常有Tech Blog分享自己的工作流：我是如何基于xxx harness搭建自己的workflow的，例如，使用GPT-5.6 sol做plan、Kimi K2.7 Coding-Highspeed编码、GPT-5.6 Luna进行代码审计、Claude Opus 4.8编写测试、Deepseekv4-flash执行测试并编写报告等。对于workflow的不同阶段区分应该调用的模型API，已经成为了多数Agent用户的共识。总的来说，涉及Design与Plan的部分，使用能力更强的模型，对于Execution与Test的部分，使用相对较弱的模型。

目前市面上terminal-based的agent harness呈井喷之势，常见的包括：Claude Code、Codex、Deepseek Harness、Kimi Code、Workbuddy、Openclaw、Opencode、Hermes等等。不同harness之间的指令集各有不同，例如，Kimi Code查看session list的命令为/sessions，而Claude Code与DSH都是/resume。

因此，当用户尝试在一个长程任务中切换模型或harness时，TA不得不选择手动管理这些步骤。例如通过/model配置当前session选择的模型。值得一提的是，切换既有session会话的模型是不推荐的，因为不同模型的层数、维度、注意力机制、位置编码可能不同，这样的行为会引发既有KV Cache的失效，进而触发已有历史的Prefill重算，用户的Token消耗在了每一次模型的冗余重算过程中。因此大多数用户的做法是新开一个session，并阅读上一任agent留下的历史摘要，最后切换合适的模型进行执行。

这无疑引入了巨量的额外操作：用户需要通过配置skill或显式地告知上一任agent在工作时留痕并提取摘要、新开session并完成配置（模型选择、Effort/Reasoning级别选择等）、通过skill或显式告知该任agent阅读上一任留下的摘要（这个过程中还需要指定摘要路径，手动管理摘要的位置与层级结构），然后下一任agent才能开始任务。更令人厌烦的是，当客户端失去了与远程服务器的连接时，这些terminal-based的session是无法持久化的，（除非放到tmux等持久会话软件中，这引入了额外的开销）用户不得不重新多开terminal并恢复这些session，才能从断点处继续运行先前的任务。

而这仅仅是同一套harness下的agent与模型API管理，跨harness的组织架构中，不同harness的agent之间的通信成为了问题，用户不得不选择中间件持久化不同harness下agent的输出，依次作为缓存层以供下游agent读取并理解。此外，不同agent对于历史会话、模型API元数据管理等机制的设计各有不同。用户需要频繁徘徊于不同harness中，增加了多agent协作管理的成本。

因此，我们希望能有一套统一的、仅提供可编辑的逻辑视图，将底层的元数据管理、数据通信与传输、配置管理等细节隐藏起来（或客制化并持久化）的多agent harness系统，协助用户管理并集成本地的不同harness软件。理想中，用户只需要基于其本地任务场景，提供workflow逻辑上的拓扑视图，即可通过流水线运行整个workflow。

因为不同agent之间的协作模式类似于蜜蜂家族中工蜂的工作方式，因此我们将这套系统命名为Workerbee，希望多个agent能像工蜂一样协同合作，帮助用户完成工作。

Design

Workflow逻辑视图

Workerbee最首要的功能包括两个，一个是隐藏不同harness的具体operation细节，另一个就是为用户提供一个可编辑的逻辑workflow拓扑图。本节主要描述逻辑视图的主要功能。

Workflow节点

我们将一个可单独通过模型API伺服的session称为一个节点。缩句：将一个session称为一个节点，也就是节点的本质是一个session，无论该session通过怎样的harness管理、保存的是怎样的模型的响应。

每个节点在逻辑拓扑图中包含以下元数据（Pre-Define）：

1. Model Preferences：一个三元组的列表。三元组为：
(model_name, context_len, retry_iters)，列表按三元组的优先级进行排序，优先级的定义由用户决定。该组元数据的作用在于：决定当前节点使用哪个模型进行伺服，该模型支持的上下文长度为context_len，当它无法提供服务时，至多指数级后退retry_iters次，之后切换到除此preference之外优先级最高的settings上。必选项，且至少有一个元素。
2. Model API Configs：与Model Preferences按优先级组成一一对应关系，也即1中对应的API Key配置。考虑到不同harness的登录方式不同，（例如Kimi Code在本地只能通过OAuth登录，而Claude Code能够指定Base URL以及API Key提供服务）
Model API Configs会提供不同类型的配置和set方式，另外各厂商或开发者可以编写插件或配置，使得harness能够接入workflow。必填项。
3. Harness Preferences：一个列表，与Model Preferences元素按优先级组成一一对应关系，也即应用哪套Harness到当前节点上，必填项。
4. References：两个不同的列表组成的tuple。第一个列表是topology结构的in，第二个是out，都是一个tuple组成的元组，三元组为（node_id, node_name, desc）。desc为一个字符串描述，描述这些前向节点与后向节点与该node的关系（可选，可AI生成）这一项在用户编辑完成后由系统完成。

5. Skills：客制化的Skill文档，是一个文档的列表。之所以需要这个文档，是因为用户可能需要客制化每个节点的具体执行逻辑。例如用户可能不希望模型使用sleep 360; 这样的命令来等待当前任务执行完并响应（或输出给下一个模型），或者该节点只能在指定的资源范围内执行（例如XPU/内存/磁盘等）或自动监控并识别存量资源然后在可用范围内执行。由于不清楚用户具体可能有什么需求，所以不会把Skill里面的内容单独做成元数据项。可选。
6. MCP：一套工具的列表，用户可以额外自定义工具并附上工具说明（输入、输出、适用场景等），可选。

每个节点维护以下成员结构（RunTime）：

1. Pending List：考虑到一条workflow流水线上可以重复提交任务，因此用户可以连续发射任务到workflow上形成多任务流水线。
2. Activate：一个0-1 Flag，用户可以选择激活或关闭某一个节点，该节点将撤回已有request停止serving。该节点前向依赖的节点将称为该节点后向节点的依赖项（需要回路检查？）

Workflow编辑

新建Workflow

用户有四种方式新建一个workflow

1. 手动编辑拓扑结构：用户自定义节点以及节点之间的依赖关系，在创建节点时，必须填写节点元数据中的必填项，可以选择选填项，或者使用AI生成这些选填项（支持与AI chat/交互后生成）。
2. AI基于任务摘要生成拓扑结构：使用AI助手基于任务摘要生成与1中结构一致的拓扑结构，AI必须生成必填项，根据任务描述选择合适的选填项填写（例如Skills/MCP）等。
3. Topology Graph Capture：开启该选项后，用户/AI助手将选择合适的Base Model开始执行接下来发射的任务，执行完成后AI助手将基于Base Model的plan与推理过程捕获这个任务执行时遵循的Workflow拓扑图，并生成一个Workflow模板。
4. 从一个Workflow模板生成Workflow：用户可以从收藏的Workflow模板中（这个模板包括了节点的所有元数据定义，不仅仅是拓扑依赖关系）实例化一个Workflow。

编辑Workflow

用户可以编辑Workflow的以下配置：

1. 从任意节点暂停/删除/恢复正在执行/处于pending状态/正在暂停的已发射任务：
 - a. 如果是暂停任务，上游对应已经pass的前置任务需要显示出下游任务已经暂停
 - b. 如果是删除任务，上游对应已经pass的前置任务也需要删除，下游对应正在执行/pending/暂停的任务也要删除，同时清空节点管理的进程/线程，并释放对应资源。但是已经执行完毕的任务链需要持久化状态（暂定不可使用这个状态进行恢复，但是需要持久化）
 - c. 如果是恢复正在执行的任务，上游对应已经显式下游正处于暂停的已经pass的任务需要恢复下游任务正在运行/pending的状态
 - d. 理论上，一个节点没有完成执行（即便它将正在执行的任务放进后台管理），它将不会通知下游节点开始执行，因此不需要考虑某个上游正在执行节点被暂停也需要暂停对应的下游节点（因为不存在对应的正在执行的下游节点）
2. Activate/Deactivate任意一个节点，将改变Workflow的拓扑结构：
 - a. Deactivate节点时，所有依赖于该节点的下游相邻节点将依赖于所有该节点依赖的上游相邻节点。
 - b. Activate时，若上游相邻节点处于activate状态，则依赖这些节点，若其中有节点deactivate，则依赖于这些节点的上游相邻节点，这个规则递归使用；该节点的所有下游相邻节点将重新依赖该节点。
3. 任意节点的元数据，但这一行为可能导致workflow中的环节由于节点的重新配置（例如切换harness/模型/API配置）失败而导致重试或中断。

删除Workflow

删除Workflow将导致所有节点从下游反向清空状态、释放资源。但该Workflow的初始定义以及Runtime状态都将被持久化。不过与删除已发射任务相同的是，用户无法从这个持久化结果中恢复Workflow。

Workflow管理

1. (de)activate节点：已经描述过规则。
2. 对一个workflow发射新的任务：对workflow输入一个新的任务，系统需要判断拓扑图入口，并开始执行或pending这个任务。

3. 调度pending-list中的任务：用户可以调度正在pending（不包括已经删除/暂停/运行/完成的历史任务）的历史任务，调整其优先级。但一旦这个任务进入删除/暂停/运行/完成状态，将无法调度。
4. 通过\btw与正在执行任务的agent交互：用户可以attach到任意一个节点查看其正在具体执行的任务，并通过\btw（by the way）或手动打断这个任务与agent交互，改变这个任务的执行过程。但仅限于正在running的任务，不包括已经删除/暂停/pending/完成的历史任务。
5. workflow历史任务管理：能够查看已经完成/删除/暂停/pending/运行状态下的所有Workflow历史。
 - a. 删除的历史任务：需要显示具体是从哪个节点发起的删除指令
 - b. 暂停的历史任务：需要显示具体是从哪个节点发起的暂停指令
 - c. pending/running的历史任务：需要显示上游已经完成的情况，以及正在pending/running的任务。
6. 多Workflow管理：用户可以同时创建多个Workflow并任意选择Workflow发射任务，但多个Workflow之间类似于大部分agent harness一样，无法进行交互。多个Workflow可以遵循相同的配置，也可以使用不同的配置。
7. Workflow模板创建与收藏：用户可以通过Graph Capture或从Pre-Define（不含Run time）的Workflow生成模板，并收藏管理这些模板。（收藏后可编辑、删除这些模板）

可选客户端

1. web：这是我认为的主力客户端。系统启动一个web应用进程，用户可以通过图形化界面管理多个Workflow、发射任务、新建/编辑/删除Workflow。
2. terminal：次选项。设计上暂定。

不同harness接入

系统应该通过插件/适配器接入不同的官方或第三方agent harness，不同harness的配置与启动方式不同，应该以明确的形式规定下来。用户在客户端可以加入自己已有的harness，并声明这些harness的可执行文件路径/环境变量。以供创建/编辑Workflow时选择。

API配置

用户可以在一个页面单独配置自己的API，包括base_url/API key等，以供创建/编辑Workflow时选择。

Base Assistant管理

在搭建Workflow时，可能会出现需要AI Assistant的情况，用户可以指定一个简单的base assistant（或者该AI助手也可以配置成一个Workflow，但这只能由用户自己配置），启动服务，并使用该AI Assistant协助自己搭建Workflow。（例如创建Workflow的2/3两种方式，都可能需要AI Assistant协助，而一些Workflow节点的元数据管理上也可能需要AI的建议或操作）

额外工具管理

主要针对Skills与MCP工具，可以理解为额外工具库，用户在搭建Workflow时可以临时自定义，或从额外工具库中选择已有的工具。

Concern

1. Workflow节点的(de)activate规则，是否会导致出现环？或者其他逻辑问题？
2. 删除/暂停一个已经发射的任务，会对它的上游节点/下游节点造成怎样的影响，目前的规则会导致资源泄露吗？或者说，当用户试图删除/暂停一个任务时，它应该遵循怎样的垃圾回收与释放资源的规则？如何确保这个过程的安全性？
3. 不同Workflow节点可能使用不同模型/Harness，这些Session之间应该如何通信，系统prompt如何构造？如何让下游节点拿到上游节点的输出？这份输出是可读的还是可写的？如果是可写的应该是让下游节点做cow还是同步？下游节点是否有机会向上游节点反馈自己的问题？依赖于同一个上游节点的下游节点应该通过barrier同步任务发布/执行时间，还是异步执行？
4. 如何处理非yolo/autopilot/bypass permission模式下AI对用户的检查请求？例如，一个workflow节点的agent session试图执行recursive delete，如何知会用户这一请求？